IoT & ELECTRONICS

Complete Domain Learning Guide

A practical guide to connected-system architecture, electronics, sensors, microcontrollers, communications, edge/cloud design, data pipelines, reliability, security, and deployment.

How to use this guide
Study each module, work through the calculations, trace the examples, and answer the checkpoints without looking at the answers. Follow electrical safety precautions and check component voltage/current limits before wiring real hardware.

Assessment target: 70% — at least 14 correct answers out of 20. Retakes can be used to learn from mistakes.

Module 1 — Introduction to IoT Systems

The four layers every connected product shares

1.1 What is IoT?

The Internet of Things (IoT) connects physical objects—such as meters, machines, wearables, vehicles, and appliances—to software systems. A device senses the world, processes measurements, communicates data, and may receive commands. A useful IoT product combines hardware, connectivity, software, operations, and a clear user benefit.

Worked example: Example: a greenhouse monitor measures temperature and soil moisture, sends readings to a platform, and lets a grower view trends or trigger irrigation.

Key takeaway: IoT is not simply a sensor connected to Wi-Fi; the value comes from turning reliable data into useful decisions or actions.

1.2 The four-layer model

A practical model has four layers. Device: sensors, actuators, microcontroller and local firmware. Network: wired or wireless links that carry messages. Platform: ingestion, identity, storage, processing, rules and device management. Application: dashboards, reports, alerts and workflows used by people or other systems.

Worked example: Greenhouse mapping: soil-moisture sensor and controller (device) → Wi-Fi or LoRaWAN (network) → cloud time-series store and rules (platform) → mobile dashboard and irrigation alert (application).

Key takeaway: Some architectures use different layer names, but clearly identify where sensing, communication, data processing and user value happen.

1.3 Edge devices, telemetry, actuators

An edge device operates near the physical process and may filter, aggregate, or react locally. Telemetry is automatically collected and transmitted measurement data, typically with timestamp and device identity. An actuator converts a control signal into physical action, such as moving a valve, spinning a motor, or switching a relay.

Worked example: A temperature sensor produces telemetry; a controller at the edge checks a threshold; a relay drives a fan actuator.

Key takeaway: Sensing observes the environment; actuation changes it. A device can do both.

1.4 Where value is created

Value can come from reduced manual inspection, earlier fault detection, better energy use, safer operation, improved quality, or automated control. Good design also accounts for installation, maintenance, privacy, reliability, and total cost—not just whether the prototype works.

```
A water tank monitor creates value when it prevents overflow and alerts a caretaker before the tank runs dry.
```

Key takeaway: Start with the decision or action a user needs, then decide which measurements and system capabilities are actually necessary.

Module checkpoint

What are the four layers in the model used here?

Answer: Device, network, platform, and application.

What is telemetry?

Answer: Automatically collected measurements transmitted from a remote source.

What does an actuator do?

Answer: Converts a control signal into physical action.

Module 2 — Electronics Fundamentals

Voltage, current, resistance, and reading a circuit

2.1 Voltage, current, resistance

Voltage is electrical potential difference, measured in volts (V). Current is the rate of charge flow, measured in amperes (A). Resistance opposes current, measured in ohms (Ω). A simple analogy is water pressure, flow, and a restriction in a pipe; it is useful conceptually but not a complete model of electrical behaviour.

```
Ohm's law: V = I × R. Rearranging gives I = V/R and R = V/I.
```

Key takeaway: Always include units. A value of 0.02 A is 20 mA.

2.2 Worked Ohm's law examples

If a 5 V source is applied across a 1 kΩ resistor, current is I = 5/1000 = 0.005 A = 5 mA. If a circuit draws 20 mA through a 220 Ω resistor, voltage across it is V = 0.02 × 220 = 4.4 V. Real components have tolerances and may not behave as ideal resistors.

```
LED current-limiting example: R ≈ (Vsupply − VLED) / I. For 5 V supply, 2 V LED drop, and 0.01 A target: R ≈ 300 Ω. A nea
```

Key takeaway: Never connect an LED directly to a supply without suitable current limiting unless the circuit/module already includes it.

2.3 Reading schematics

A schematic represents electrical connections with symbols and nets. Identify the supply, ground, components, polarity, and pin labels. Components connected to the same net are electrically connected even if the line bends or is drawn in different places. A breadboard's internal rows and power rails have specific connection patterns that should be checked for the board being used.

```
Before powering a circuit, trace: supply → load/component → return/ground. Confirm that the expected current path is co
```

Key takeaway: A schematic is a logical map, not necessarily a physical layout. Double-check pin numbering and component orientation.

2.4 Multimeter safety

A multimeter measures voltage, current, resistance and often continuity. Voltage is generally measured in parallel across two points. Current is measured in series with the circuit and may require moving the probe to a current input jack. Resistance/continuity should be measured on a de-energised circuit. Start on a suitable range and inspect the meter's ratings.

```
For a low-voltage battery circuit: verify the meter is in voltage mode, use the correct input jack, place probes across s
```

Key takeaway: Never place a meter configured for current directly across a voltage source; that can create a short. Do not probe mains voltage unless trained and equipped for it.

Module checkpoint

A 12 V supply is across 2 kΩ. What current flows ideally?

Answer: I = V/R = 12/2000 = 0.006 A = 6 mA.

How is voltage measured with a multimeter?

Answer: In parallel across the two points, using the correct voltage mode and input.

Should resistance be measured on a powered circuit?

Answer: No; de-energise the circuit first.

Module 3 — Sensors & Signal Conditioning

Getting a trustworthy number out of the physical world

3.1 Selecting sensors

Select a sensor by measured quantity, range, accuracy, resolution, response time, environmental limits, power draw, calibration needs, and interface. Accuracy describes closeness to the true value; resolution describes the smallest distinguishable change; precision/repeatability describes how consistent repeated readings are. A sensor can be precise but systematically wrong.

Worked example: A room-temperature sensor with a range of –10 to 60 °C may suit indoor monitoring but not a high-temperature industrial process.

Key takeaway: Choose the sensor for the real operating range and environment, not just price or availability.

3.2 Analog signals and ADC

An analog signal varies continuously. An analog-to-digital converter (ADC) samples a voltage and represents it as a digital number. For an ideal N-bit ADC, there are 2^N possible codes. With a 12-bit ADC, codes range from 0 to 4095. If the reference is 3.3 V, an approximate conversion is voltage ≈ code × 3.3/4095, subject to ADC design and calibration.

Worked example: Example: code 2048 on an ideal 12-bit, 3.3 V ADC corresponds to about 1.65 V.

Key takeaway: Check the microcontroller's allowed input voltage. A 5 V sensor output can damage a 3.3 V-only input without appropriate level conversion.

3.3 Sampling rate and aliasing

Sampling rate is the number of measurements per second, measured in hertz (Hz). Slowly changing temperature can be sampled infrequently; vibration or audio requires much faster sampling. To represent a band-limited signal, the sampling rate must exceed twice its highest frequency, with practical margin and suitable anti-alias filtering. Sampling too slowly can make a changing signal appear misleadingly slow or stable.

```
A sensor sampled once every 10 seconds yields 0.1 samples/second. It may suit room temperature trends but miss a short v
```

Key takeaway: Match the sample rate to the dynamics of the physical signal and the purpose of the measurement.

3.4 Calibration, filtering, and noise

Calibration compares sensor readings with a known reference and determines corrections or a mapping. Filtering reduces unwanted variation, for example with a moving average. Filters introduce trade-offs: smoothing can reduce noise but also delay or hide genuine rapid changes. Record calibration conditions and repeat calibration when required by drift or operating conditions.

```
For readings 20, 21, 19, 20, 30 °C, a five-value mean is 22 °C; the spike affects the average. A median filter would retu
```

Key takeaway: Filtering cannot fix every sensor fault. Distinguish noise, bias, drift, saturation, and genuine events.

Module checkpoint

What does an ADC do?

Answer: Converts an analog input into a digital code.

Why calibrate a sensor?

Answer: To compare against a known reference and correct measurement error.

What risk comes from sampling too slowly?

Answer: Aliasing or missing rapid events.

Module 4 — Microcontrollers & GPIO

The small computer that runs the device

4.1 Microcontrollers and firmware

A microcontroller combines a processor, memory, timers, communication interfaces, and GPIO peripherals on one chip. Firmware is the code running on it. Compared with a general-purpose computer, a microcontroller usually has tighter limits on memory, power, timing, and hardware interfaces.

```
Typical development boards include ESP32 and Arduino-compatible boards. Confirm the exact chip's voltage, pin mapping, an
```

Key takeaway: A development board's labels and capabilities depend on the model; not every pin supports every function.

4.2 GPIO inputs and outputs

General-purpose input/output (GPIO) pins can be configured as digital inputs or outputs. An input reads a logic level; an output drives a logic level within the pin's electrical limits. A floating input can change unpredictably, so use an appropriate pull-up or pull-down. Buttons often need debouncing because contacts can produce rapid transitions.

```
Arduino-style example:
const int ledPin = 2;
void setup() { pinMode(ledPin, OUTPUT); }
void loop() {
  digitalWrite(ledPin, HIGH);
  delay(500);
  digitalWrite(ledPin, LOW);
  delay(500);
}
```

Key takeaway: Check board-specific pin numbers and avoid driving loads that exceed GPIO current ratings. Use a driver circuit for motors, relays, and other higher-current loads.

4.3 Interrupts and timers

An interrupt allows the processor to respond to an event without repeatedly polling for it. Interrupt service routines should be short and predictable; defer lengthy work to the main loop. Timers support periodic actions and precise intervals. Shared data between interrupt and main code requires care, including appropriate atomic or volatile usage depending on the platform and data type.

```
A pulse counter may use an interrupt to record edges while the main loop reports counts periodically.
```

Key takeaway: Do not perform long delays, heavy logging, or unsafe blocking operations inside an interrupt handler.

4.4 PWM and predictable firmware loops

Pulse-width modulation (PWM) switches a signal on and off rapidly and varies its duty cycle. A 25% duty cycle means the signal is high for about one quarter of each period. PWM can control LED brightness or motor-driver input; it is not a substitute for a suitable power driver. A structured loop reads inputs, validates/filters them, updates state, and applies outputs.

```
Duty cycle = (on time / period) × 100%. If the period is 10 ms and on-time is 2 ms, duty cycle is 20%.
```

Key takeaway: Use non-blocking timing patterns when the device must respond to multiple events. Include safe output states during startup and faults.

Module checkpoint

What does GPIO mean?

Answer: General-purpose input/output.

Why use a pull-up or pull-down resistor?

Answer: To give an input a defined level rather than leaving it floating.

What is PWM duty cycle?

Answer: The fraction of each period that the signal is high.

Module 5 — Communication Protocols

How devices talk to each other and to the cloud

5.1 I2C, SPI, and UART

I2C typically uses SDA and SCL, supports addressed peripherals on a shared bus, and requires pull-up resistors. SPI commonly uses clock, controller-out, controller-in, and chip-select signals; it is often fast but uses more wires. UART is asynchronous serial communication using TX and RX with agreed settings such as baud rate. Electrical levels must also be compatible.

```
I2C is common for sensors on a board; SPI is common for displays or high-throughput peripherals; UART is common for seria
```

Key takeaway: These are board-level interfaces, not automatically internet protocols. Check voltage levels, bus speed, wiring length, and device addressing.

5.2 Wi-Fi, BLE, and LoRaWAN

Wi-Fi provides comparatively high throughput and is useful where local network access and power are available. Bluetooth Low Energy (BLE) supports short-range, low-power communication, often to a phone or gateway. LoRaWAN targets long-range, low-data-rate communication with low-power end devices. Selection depends on coverage, payload size, battery life, latency, infrastructure, and cost.

Worked example: A camera may need Wi-Fi bandwidth; a fitness sensor may use BLE; a remote water meter may send small readings over LoRaWAN.

Key takeaway: Do not choose by range alone: radio conditions, buildings, antenna design, regional rules, and network availability matter.

5.3 MQTT publish-subscribe messaging

MQTT is a lightweight messaging protocol often used by IoT systems. Publishers send messages to topics through a broker; subscribers receive messages for topics to which they subscribe. Producers and consumers are decoupled. Quality of Service (QoS) levels trade delivery assurance against overhead; retained messages and persistent sessions have specific behaviours that should be configured deliberately.

```
Device publishes to greenhouse/zone1/moisture; dashboard subscribes to greenhouse/+/moisture. Use a defined payload such
```

Key takeaway: Protect MQTT with authenticated identities, access controls on topics, and encrypted transport where supported.

5.4 Payload size and reliability

Payload size is the data carried in a message, excluding or alongside protocol overhead depending on how it is measured. Smaller payloads reduce airtime, bandwidth and energy, especially on constrained links. Include essential units, timestamps, identifiers and schema/version information. Design retries and duplicate handling carefully because messages may be delayed or delivered more than once.

```
Instead of sending every raw sample, a node may send minimum, maximum, mean, sample count, and time window once per inter
```

Key takeaway: A compact payload should still be interpretable and safe to evolve. Avoid sending unnecessary personal or sensitive data.

Module checkpoint

Which protocol commonly uses SDA and SCL?

Answer: I2C.

How does MQTT pub-sub work?

Answer: Publishers send messages to topics through a broker; subscribers receive matching messages.

Which is suited to long-range, small payloads at low power?

Answer: LoRaWAN, when coverage and deployment conditions support it.

Module 6 — Edge vs Cloud Processing

Deciding where computation should happen

6.1 Edge computing trade-offs

Edge computing processes data on or near the device. It can reduce response latency, network usage, cloud costs and exposure of raw data. Cloud processing offers centralised management, larger compute resources, and fleet-wide analysis. Many practical systems use both.

```
A local controller stops a motor immediately when an unsafe temperature is detected; the cloud stores trends and supports
```

Key takeaway: Put time-critical safety logic close to the process, and design for safe operation when connectivity is lost.

6.2 Latency budget

Latency is the time between an event and the system's response. A latency budget assigns the maximum allowed delay across sensing, local processing, network transport, server processing and actuation. Network delay can vary, so a cloud round trip may be unsuitable for very fast control.

```
If a response must occur within 50 ms but a network round trip may take 100 ms, the critical decision cannot depend sole
```

Key takeaway: Measure worst-case or high-percentile delay, not just the average. Safety-critical systems need appropriate engineering and validation.

6.3 Store-and-forward buffering

Store-and-forward means storing readings locally while a network is unavailable and sending them after connectivity returns. A design must define buffer capacity, timestamping, ordering, expiry, retry backoff, duplicate handling, and what happens when the buffer fills. Use persistent storage if data must survive a reboot.

```
If a device records one reading per minute and may be offline for 6 hours, it needs room for at least 360 readings plus r
```

Key takeaway: Buffering prevents avoidable data loss, but cannot compensate for an unlimited outage with finite storage.

6.4 Aggregation and privacy

Aggregation combines many readings into summaries such as average, minimum, maximum, count, or percentiles. It reduces transmission and storage costs. Processing near the source can also avoid sending sensitive raw data. Preserve raw data only when needed for diagnosis, regulation, or analysis, and apply access and retention controls.

```
Send a 15-minute average and maximum instead of 900 one-second readings when detailed raw samples are not required.
```

Key takeaway: Aggregation can hide short spikes; preserve event-triggered samples when rare events matter.

Module checkpoint

What is edge computing?

Answer: Processing data on or near the device.

What is store-and-forward?

Answer: Buffering data during network outages and transmitting it later.

Why aggregate readings?

Answer: To reduce data volume, bandwidth and storage use.

Module 7 — IoT Data Pipelines

From a reading to a dashboard or alert

7.1 Time-series data and ingestion

Time-series data consists of timestamped measurements, often indexed by device, metric, and time. Ingestion receives messages, authenticates or validates them, normalises units and timestamps, and writes them to storage or a stream processor. Consider late, duplicated, missing, or out-of-order readings.

```
Example record: device_id, timestamp_utc, metric, value, unit, firmware_version. A moisture value without a unit or times
```

Key takeaway: Validate schema and identity at ingestion; separate event time (when measured) from arrival time (when received).

7.2 Storage, dashboards, and retention

A time-series database or suitable relational store can support queries over time ranges and aggregations. Dashboards should show units, time windows, device status, and missing data clearly. A retention policy specifies how long high-resolution readings remain before aggregation or deletion, balancing cost, usefulness and legal or operational needs.

```
Keep raw one-second data for 7 days, 1-minute summaries for 90 days, and hourly summaries for longer-term trend analysis-
```

Key takeaway: Do not imply that a blank graph means zero. Distinguish zero values from missing measurements and disconnected devices.

7.3 Threshold alerts without alert fatigue

A threshold alert triggers when a value crosses a boundary. Simple alerts may trigger too often because of noise or values oscillating near the boundary. Improve them with hysteresis (different trigger and clear levels), persistence windows, rate limits, device health checks, and severity levels. Choose thresholds from operational requirements, not arbitrary guesses.

```
Alert if temperature remains above 40 °C for 3 minutes; clear the alert only after it drops below 37 °C. This reduces rap
```

Key takeaway: Test thresholds against real and edge-case data. A threshold should lead to an action someone can take.

7.4 End-to-end pipeline

A useful pipeline is: device measurement → validation and timestamping → secure transport → ingestion → durable storage → rules/analytics → dashboard or notification → acknowledgement/action. Monitor pipeline health as well as the physical measurement: device last-seen time, failed ingestion, backlog, and alert delivery.

```
If no readings arrive for 30 minutes, a separate device-offline alert may be more useful than showing the last value as i
```

Key takeaway: A dashboard is only trustworthy when data freshness, device state, and measurement quality are visible.

Module checkpoint

What is time-series data?

Answer: Timestamped measurements typically queried by time and aggregation.

Why use hysteresis in threshold alerts?

Answer: To prevent repeated triggering and clearing near one boundary.

Why distinguish missing data from zero?

Answer: A missing measurement does not mean the physical value was zero.

Module 8 — Power & Reliability

Designing devices that keep running unattended

8.1 Power budget

A power budget estimates average current or power across operating modes and helps size a battery. Average current is the sum of each mode's current multiplied by the fraction of time spent in that mode. Battery life is only an estimate because usable capacity depends on temperature, discharge rate, ageing, regulator efficiency, radio conditions and battery chemistry.

Worked example: Example: device uses 20 mA for 10 seconds each hour and 0.1 mA for the remaining 3590 seconds. Average current ≈ (20×10 + 0.1×3590)/3600 ≈ 0.155 mA.

Key takeaway: Include sensors, regulators, radio bursts, LEDs, and sleep current. A nominal battery capacity divided by average current gives only a first estimate.

8.2 Duty cycling and sleep modes

Duty cycling keeps a device in a low-power state most of the time and wakes it to measure, process, and transmit. Reducing radio-on time often matters greatly. The design must consider wake-up time, sensor warm-up, clock drift, network rejoin cost, and whether the application can tolerate delayed readings.

Worked example: A soil monitor wakes every 15 minutes, samples moisture, sends one compact packet, and returns to sleep.

Key takeaway: More frequent measurements improve responsiveness but typically use more energy. Choose the interval based on how quickly the underlying condition changes.

8.3 Watchdog timers and graceful recovery

A watchdog timer resets or recovers a device if firmware fails to make expected progress. Feed the watchdog only after meaningful progress, not blindly from a timer that keeps running while the main task is stuck. After reset, firmware should initialise safely, recover configuration, and report reboot reasons when possible.

Worked example: A controller that fails to complete its main cycle within the watchdog interval is restarted; outputs should default to a safe state during restart.

Key takeaway: Watchdogs address some hangs, not every failure. Add brownout handling, input validation, bounded retries, and recovery testing.

8.4 MTBF and reliability planning

Mean Time Between Failures (MTBF) is a statistical reliability measure, commonly used for repairable systems under stated assumptions. It is not a promise that each unit will operate for that exact duration. Reliability also depends on environmental stress, connectors, moisture, thermal cycling, component quality, firmware defects, and maintenance access.

```
For a large fleet, even a low per-device failure rate can produce regular field service work. Plan spare parts, diagnosti
```

Key takeaway: Use field data to revise reliability estimates. Test environmental extremes and likely failure modes before large deployment.

Module checkpoint

How is average current estimated across modes?

Answer: Multiply each mode's current by its time fraction, then sum the contributions.

What does a watchdog do?

Answer: Resets or recovers a device when expected progress stops.

Is MTBF a guarantee for each device?

Answer: No; it is a statistical reliability measure under assumptions.

Module 9 — Device Security

Connected devices are an attack surface

9.1 Device identity and provisioning

Every device should have a unique identity and credentials provisioned securely. Avoid shared default passwords and hard-coded fleet-wide secrets. Provisioning associates a device with its owner, permissions and platform records. Credentials should be protected at rest, rotated or revoked when needed, and handled through a controlled manufacturing and onboarding process.

Worked example: A platform grants a device permission to publish only to its own telemetry topic rather than to every topic in the fleet.

Key takeaway: Unique identity makes revocation, auditing, and limiting the impact of a compromised device possible.

9.2 Encryption in transit and at rest

Transport encryption, commonly using TLS where applicable, helps protect data from eavesdropping and tampering during communication. Encryption at rest protects stored data when storage media or backups are exposed. Encryption does not replace authentication, authorisation, key management, secure endpoints, or careful logging.

```
Use certificate validation and appropriate authentication when connecting to a broker or API; do not disable verificatio
```

Key takeaway: A secure design includes key storage, certificate expiry handling, access control, and recovery from credential compromise.

9.3 Secure boot and signed updates

Secure boot verifies that the firmware being executed is trusted, often using a cryptographic signature rooted in protected trust material. Over-the-air (OTA) updates allow remote firmware installation. Update packages should be authenticated, integrity-checked, compatible with the device, and recoverable if installation fails. Rollback protection may be needed to prevent reinstalling vulnerable firmware.

Worked example: A device downloads an update, verifies its signature, installs it into an inactive slot, boots it, checks health, and commits or rolls back.

Key takeaway: Never treat a successful download as proof that firmware is trustworthy. Plan interrupted downloads, power loss, failed boot and recovery.

9.4 Defence in depth

Device security includes secure development, minimal exposed services, least privilege, signed firmware, protected secrets, encrypted communications, logging, vulnerability response, and an end-of-life policy. Updates need authentication and a secure recovery path. Physical access may also expose debug ports or storage, depending on the design.

Worked example: A fleet manager can disable a lost device, revoke its credentials, deploy a fix, and monitor whether each device successfully updated.

Key takeaway: Security is a lifecycle responsibility from design and manufacturing through operation and decommissioning.

Module checkpoint

Why should each device have a unique identity?

Answer: It supports authentication, individual permissions, revocation and auditing.

What does secure boot verify?

Answer: That firmware is trusted, typically using signature verification.

Why plan OTA rollback?

Answer: To recover safely if an update fails or the new firmware cannot boot.

Module 10 — From Prototype to Deployment

What changes when you leave the workbench

10.1 Enclosures and ingress protection

An IP (Ingress Protection) rating describes enclosure protection against access to hazardous parts and ingress of solids and water under specified tests. The first digit relates to solids/dust; the second relates to water. A rating does not automatically guarantee resistance to every chemical, corrosion, UV exposure, impact, or temperature range.

Worked example: A device exposed to rain, dust and cleaning procedures needs an enclosure and connectors rated for those conditions, including cable entries and seals.

Key takeaway: Choose the rating from the actual environment. Openings, connectors, seals and installation can reduce real-world protection.

10.2 Field trials and environmental testing

A field trial is a limited deployment in real operating conditions before broad release. It reveals installation issues, wireless dead zones, sensor drift, user behaviour, maintenance burden, and environmental problems that bench tests may miss. Define success metrics, a test duration, support procedures, rollback, and a way to collect feedback.

Worked example: Pilot 10 units across different sites, compare readings against a reference, record disconnects and battery use, and interview operators before expanding.

Key takeaway: A successful demonstration on one workbench is not proof of reliable fleet performance.

10.3 Fleet management and observability

Fleet management supports remote inventory, configuration, health monitoring, firmware updates, and decommissioning. Track last-seen time, firmware version, battery level, connectivity quality, reset count, sensor faults, and update status. Alerts should distinguish device failure from a real environmental event.

Worked example: A fleet dashboard groups devices by firmware version and flags devices that have not checked in or failed an update.

Key takeaway: Plan how to find, diagnose, update, and retire a device without requiring a technician to visit every location.

10.4 Bill of materials and total cost

A bill of materials (BOM) lists components and unit costs required to build one product. Total cost also includes assembly, enclosure, testing, packaging, shipping, cloud services, connectivity, installation, maintenance, support, certification and replacements. Component availability and substitutions matter for long-lived products.

Worked example: If components cost ■1,200, enclosure ■250, assembly/test ■300, and packaging ■100, direct unit build cost is ■1,850 before logistics, cloud and support.

Key takeaway: Use realistic volume pricing and include recurring operating costs. A low-cost prototype may be expensive to maintain at fleet scale.

Module checkpoint

What do the two main digits in an IP rating describe?

Answer: Protection against solids/dust and water ingress, respectively.

What is a field trial for?

Answer: Finding issues under real conditions before wider deployment.

What should a BOM include?

Answer: The itemised components and their costs; total cost analysis should also include operations and support.

Practice Assessment — 20 Questions

Choose one best answer for each question. Award one point per correct answer. A score of 14/20 (70%) meets the passing threshold.

1. Which sequence best represents the four-layer IoT model?

A. Device, network, platform, application

B. Keyboard, monitor, printer, mouse

C. Compile, link, boot, print

D. Voltage, resistance, database, browser

2. What is telemetry?

A. A physical actuator

B. Automatically collected and transmitted measurements

C. A battery chemistry

D. A firmware compiler

3. A 10 V supply is connected across a 2 kΩ resistor. Ideal current is:

A. 20 A

B. 5 A

C. 5 mA

D. 0.2 mA

4. How should a multimeter measure voltage across a resistor?

A. In series with the resistor

B. In parallel across the resistor

C. With probes shorted together

D. In resistance mode on a powered circuit

5. What does an ADC do?

A. Converts analog signals to digital values

B. Converts MQTT to Wi-Fi

C. Stores firmware updates

D. Generates mechanical motion

6. Why is calibration performed?

A. To increase radio range automatically

B. To compare measurements with a known reference and correct error

C. To encrypt telemetry

D. To reduce file size

7. What is a floating GPIO input likely to do?

A. Always read zero

B. Change unpredictably due to noise

C. Become an actuator

D. Increase battery capacity

8. What does a 30% PWM duty cycle mean?

A. Signal is high for about 30% of each period

B. Device is 30% accurate

C. Battery is 30% charged

D. Data is lost 30% of the time

9. Which bus commonly uses SDA and SCL?

A. UART

B. I2C

C. MQTT

D. LoRaWAN

10. Which protocol is designed for low-power, long-range, small-message IoT links?

A. HDMI

B. LoRaWAN

C. USB display protocol

D. HTTP always

11. In MQTT, who routes messages from publishers to subscribers?

A. ADC

B. Broker

C. Actuator

D. Comparator

12. Why process a safety threshold at the edge?

A. To depend on a cloud round trip

B. To reduce response latency and continue through connectivity loss

C. To avoid needing sensors

D. To increase payload size

13. What does store-and-forward do during a network outage?

A. Deletes all readings

B. Buffers data and sends it after connectivity returns

C. Replaces sensor calibration

D. Disables device identity

14. What is time-series data?

A. Measurements with timestamps

B. A list of passwords

C. An unconnected actuator

D. A static schematic only

15. What can hysteresis help prevent?

A. Every sensor error

B. Repeated alert triggering near a threshold

C. All network outages

D. Battery ageing

16. What is duty cycling primarily used for?

A. Saving energy by sleeping between tasks

B. Increasing enclosure IP rating

C. Creating a unique device certificate

D. Increasing ADC bit depth

17. What is a watchdog timer for?

A. Measuring water ingress

B. Recovering from firmware that stops making expected progress

C. Encrypting MQTT messages

D. Calibrating a sensor automatically

18. Which practice best supports secure device identity?

A. Use one shared default password for all units

B. Use unique credentials and controlled provisioning

C. Disable authentication

D. Publish credentials in telemetry

19. What does secure boot aim to ensure?

A. Only trusted firmware is executed

B. Every sensor reports zero

C. All messages are public

D. The battery never degrades

20. What is a field trial?

A. A purely simulated test with no deployed hardware

B. A limited real-world deployment before broad release

C. A method to calculate Ohm's law

D. A type of ADC

Answer Key with Explanations

1. Correct answer: **A** — The model separates physical device functions, communication, platform processing, and user-facing applications.

2. Correct answer: **B** — Telemetry carries measurements from a remote source for monitoring or analysis.

3. Correct answer: **C** — I = V/R = 10/2000 A = 0.005 A = 5 mA.

4. Correct answer: **B** — Voltage is measured across two points in parallel.

5. Correct answer: **A** — An ADC samples an analog voltage and represents it as a digital code.

6. Correct answer: **B** — Calibration helps align sensor output with a known reference.

7. Correct answer: **B** — A defined pull-up or pull-down level prevents an input from floating.

8. Correct answer: **A** — Duty cycle is the fraction of a period for which the signal is high.

9. Correct answer: **B** — I2C uses the serial data and serial clock lines.

10. Correct answer: **B** — LoRaWAN targets low-power wide-area communication with small payloads.

11. Correct answer: **B** — The broker receives publications and delivers messages to matching subscriptions.

12. Correct answer: **B** — Local decisions can respond quickly without waiting for a network connection.

13. Correct answer: **B** — Local buffering preserves readings when the network is temporarily unavailable.

14. Correct answer: **A** — Timestamps enable trend analysis and time-window queries.

15. Correct answer: **B** — Separate trigger and clear boundaries reduce alert oscillation.

16. Correct answer: **A** — Duty cycling reduces average energy use when frequent continuous operation is unnecessary.

17. Correct answer: **B** — A watchdog can reset a device after a hang or missed progress deadline.

18. Correct answer: **B** — Unique, protected credentials support individual authentication and revocation.

19. Correct answer: **A** — Secure boot validates firmware against a trusted signature or root of trust.

20. Correct answer: **B** — Field trials expose installation, environmental, connectivity and operational issues.

Quick Revision Sheet

IoT architecture: Device senses/acts; network transports messages; platform ingests/processes/stores; application supports decisions and workflows.

Electronics: V = I × R; measure voltage in parallel; measure current in series; measure resistance only on a de-energised circuit.

Sensors: Choose range, accuracy, resolution, interface and environment; ADC converts analog to digital; calibration uses a known reference.

Microcontrollers: GPIO reads/drives logic levels; interrupts respond to events; PWM varies duty cycle; protect pins from excessive voltage/current.

Communications: I2C uses SDA/SCL; SPI is often fast with more wires; UART uses TX/RX; Wi-Fi, BLE and LoRaWAN serve different trade-offs.

Edge/cloud: Edge supports low latency and local resilience; cloud supports central management and larger-scale analysis; buffer outages with store-and-forward.

Data pipelines: Validate and timestamp telemetry; ingest and store it; visualise with freshness indicators; use hysteresis and persistence for robust alerts.

Reliability: Estimate average current across modes; duty-cycle to save power; use watchdogs and safe recovery; MTBF is statistical, not a guarantee.

Security: Unique device identity, least privilege, encrypted transport, protected keys, secure boot, signed OTA updates and rollback.

Deployment: Check enclosure/environment, conduct field trials, manage fleets remotely, and calculate BOM plus operating/support costs.

End of guide. Review missed questions by revisiting the related module, then explain the concept in your own words or work through a fresh example.